

Java String Cheat Sheet

Essential Methods for DSA, LeetCode & Interviews

Class: java.lang.String

Implements: Comparable, CharSequence, Serializable

Properties: Final, Immutable, UTF-16

Ordering: Lexicographic (compareTo)

■ MUST KNOW METHODS

1. length()

```
String s = "Hello"; s.length(); // 5
```

Returns number of characters

Time: $O(1)$

2. charAt()

```
char c = s.charAt(1); // 'e'
```

Very common in DSA problems

Time: $O(1)$

3. substring()

```
s.substring(2); // "cdef" (from index 2) s.substring(2, 5); // "cde" [from, to)
```

Remember: [from, to) — exclusive end

Time: $O(k)$ where k = length of substring

4. equals()

```
"abc".equals("abc"); // true // NEVER use == for content comparison
```

Compares contents, not references

Time: $O(n)$

5. equalsIgnoreCase()

```
"Hello".equalsIgnoreCase("HELLO"); // true
```

6. compareTo()

```
"apple".compareTo("banana"); // negative "cat".compareTo("cat"); // 0
```

```
"dog".compareTo("cat"); // positive
```

Lexicographic comparison. Very useful for sorting

Time: $O(\min(n, m))$

7. compareToIgnoreCase()

Same as compareTo() but ignoring case

8. contains()

```
String s = "abcdef"; s.contains("cd"); // true
```

Time: $O(n)$

9. startsWith() / endsWith()

```
s.startsWith("abc"); // true s.endsWith("def"); // true
```

Time: $O(\text{prefix/suffix length})$

10. indexOf() / lastIndexOf()

```
String s = "banana"; s.indexOf('a'); // 1 s.indexOf("na"); // 2 s.lastIndexOf('a'); // 5 //  
Not found → -1
```

Search for char or substring

Time: $O(n)$

11. split()

```
String s = "a,b,c"; String[] arr = s.split(","); // Result: ["a", "b", "c"]
```

Uses regex. Be careful with special regex chars

Time: $O(n)$

12. trim()

```
" hello ".trim(); // "hello"
```

Removes leading and trailing whitespace

Time: $O(n)$

13. toCharArray()

```
char[] arr = s.toCharArray();
```

Essential for: Reverse String, Palindrome, Character Frequency

Time: $O(n)$

14. replace()

```
String s = "banana"; s.replace('a', 'x'); // "bxnxbnx"
```

Replace all occurrences of a char

Time: $O(n)$

15. replaceAll()

```
s.replaceAll("\\d", ""); // Removes all digits
```

Uses regex. Mostly avoid in DSA unless regex is intended

Time: $O(n)$

16. isEmpty()

```
s.isEmpty(); // Equivalent to s.length() == 0
```

17. valueOf()

```
String.valueOf(123); // "123" String.valueOf(true); // "true"
```

Convert primitive to String

Time: $O(n)$

18. join()

```
String.join("-", "A", "B", "C"); // "A-B-C"
```

Useful occasionally for concatenation with delimiter

Time: $O(n)$

■ COMMON CONVERSIONS

Conversion	Code
String → char[]	<code>char[] arr = s.toCharArray();</code>
char[] → String	<code>String s = new String(arr);</code>
String → int	<code>int x = Integer.parseInt(s);</code>
int → String	<code>String s = String.valueOf(x);</code>

■ SORTING STRINGS

Uses **compareTo()** internally:

```
String[] words = {"cat", "apple", "dog"}; Arrays.sort(words); // Result: ["apple", "cat", "dog"]  
// Descending: Arrays.sort(words, Comparator.reverseOrder());
```

■■ TIME COMPLEXITY TABLE

Method	Time
length	O(1)
charAt	O(1)
substring	O(k)
equals	O(n)
compareTo	O(min(n,m))
contains	O(n)
startsWith / endsWith	O(prefix/suffix length)
indexOf / lastIndexOf	O(n)
split	O(n)
trim	O(n)
replace	O(n)
toCharArray	O(n)
valueOf	O(n)

■ DSA CHECKLIST

Must Know	Good to Know	Skip for DSA
■ <code>length()</code> ■ <code>charAt()</code> ■ <code>substring()</code> ■ <code>equals()</code> ■ <code>compareTo()</code> ■ <code>contains()</code> ■ <code>startsWith()</code> ■ <code>endsWith()</code> ■ <code>indexOf()</code> ■ <code>lastIndexOf()</code> ■ <code>split()</code> ■ <code>trim()</code> ■ <code>replace()</code> ■ <code>toCharArray()</code> ■ <code>isEmpty()</code> ■ <code>String.valueOf()</code>	■ <code>equalsIgnoreCase()</code> ■ <code>compareToIgnoreCase()</code> ■ <code>join()</code> ■ <code>format()</code>	■ <code>intern()</code> ■ Unicode code-point methods ■ Regex-heavy methods (<code>matches</code>, <code>replaceFirst</code>) ■ Charset/byte conversion methods ■ Locale-specific <code>toLowerCase/toUpperCase</code>

■ ESSENTIAL: `StringBuilder`

Since **String is immutable**, efficient string construction, reversal, insertion, deletion, and concatenation in interview problems almost always relies on **StringBuilder**. It's the natural next chapter after `String`.

```
StringBuilder sb = new StringBuilder("Hello"); sb.append(" World"); sb.reverse();
sb.insert(5, "!"); sb.delete(0, 2); String result = sb.toString();
```

Reference: Oracle Java SE 8 Docs — `java.lang.String` | Compiled for DSA Interview Prep